

# Dropout (neural networks)

**Dropout** is a regularization technique for reducing overfitting in artificial neural networks by preventing complex co-adaptations on training data. The technique consists of randomly setting the inputs and/or outputs of neurons to zero. It is performed during the training process of a neural network, not during inference.<sup>[1][2][3]</sup>

A related older technique is *dilution*, which randomly decreases weights towards zero.<sup>[4]</sup>



On the left is a fully connected neural network with two hidden layers. On the right is the same network after applying dropout. 

## Dilution

Dilution is usually split in *weak dilution* and *strong dilution*. Weak dilution describes the process in which the finite fraction of removed connections is small, and strong dilution refers to when this fraction is large. There is no clear distinction on where the limit between strong and weak dilution is, and often the distinction is dependent on the precedent of a specific use-case and has implications for how to solve for exact solutions.

Sometimes dilution is used for adding damping noise to the inputs. In that case, weak dilution refers to adding a small amount of damping noise, while strong dilution refers to adding a greater amount of damping noise. Both can be rewritten as variants of weight dilution.

These techniques are also sometimes referred to as random pruning of weights, but this is usually a non-recurring one-way operation. The network is pruned, and then kept if it is an improvement over the previous model. Dilution and dropout both refer to an iterative process. The pruning of weights typically does not imply that the network continues learning, while in dilution/dropout, the network continues to learn after the technique is applied.

## Generalized linear network

Output from a layer of linear nodes, in an artificial neural net can be described as

$$y_i = \sum_j w_{ij} x_j \tag{1}$$

- $y_i$  – output from node  $i$
- $w_{ij}$  – real weight before dilution, also called the Hebb connection strength
- $x_j$  – input from node  $j$

This can be written in vector notation as

$$\mathbf{y} = \mathbf{W}\mathbf{x} \quad (2)$$

- $\mathbf{y}$  – output vector
- $\mathbf{W}$  – weight matrix
- $\mathbf{x}$  – input vector

Equations (1) and (2) are used in the subsequent sections.

## Weak dilution

During weak dilution, the finite fraction of removed connections (the weights) is small, giving rise to a tiny uncertainty. This edge-case can be solved exactly with mean field theory. In weak dilution the impact on the weights can be described as

$$\hat{w}_{ij} = \begin{cases} w_{ij}, & \text{with } P(c) \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

- $\hat{w}_{ij}$  – diluted weight
- $w_{ij}$  – real weight before dilution
- $P(c)$  – the probability of  $c$ , the probability of keeping a weight

The interpretation of probability  $P(c)$  can also be changed from keeping a weight into pruning a weight.

In vector notation this can be written as

$$\hat{\mathbf{W}} = \mathbf{g}(\mathbf{W}, c) \quad (4)$$

where the function  $\mathbf{g}(\cdot)$  imposes the previous dilution.

In weak dilution only a small and fixed fraction of the weights are diluted. When the number of terms in the sum goes to infinite (the weights for each node) it is still infinite (the fraction is fixed), thus mean field theory can be applied. In the notation from Hertz et al.<sup>[4]</sup> this would be written as

$$\langle h_i \rangle = c \sum_j w_{ij} \langle S_j \rangle \quad (5)$$

- $\langle h_i \rangle$  the mean field temperature
- $c$  – a scaling factor for the temperature from the probability of keeping the weight
- $w_{ij}$  – real weight before dilution, also called the Hebb connection strength
- $\langle S_j \rangle$  – the mean stable equilibrium states

There are some assumptions for this to hold, which are not listed here.<sup>[5][6]</sup>

## Strong dilution

When the dilution is strong, the finite fraction of removed connections (the weights) is large, giving rise to a huge uncertainty.

## Dropout

---

Dropout is a special case of the previous weight equation (3), where the aforementioned equation is adjusted to remove a whole row in the vector matrix, and not only random weights

$$\hat{\mathbf{w}}_j = \begin{cases} \mathbf{w}_j, & \text{with } P(c) \\ \mathbf{0}, & \text{otherwise} \end{cases} \quad (6)$$

- $P(c)$  – the probability  $c$  to keep a row in the weight matrix
- $\mathbf{w}_j$  – real row in the weight matrix before dropout
- $\hat{\mathbf{w}}_j$  – diluted row in the weight matrix

Because dropout removes a whole row from the vector matrix, the previous (unlisted) assumptions for weak dilution and the use of mean field theory are not applicable.

The process by which the node is driven to zero, whether by setting the weights to zero, by “removing the node”, or by some other means, does not impact the end result and does not create a new and unique case. If the neural net is processed by a high-performance digital array-multiplier, then it is likely more effective to drive the value to zero late in the process graph. If the net is processed by a constrained processor, perhaps even an analog neuromorphic processor, then it is likely a more power-efficient solution is to drive the value to zero early in the process graph.

## Google's patent

---

Although there have been examples of randomly removing connections between neurons in a neural network to improve models,<sup>[4]</sup> this technique was first introduced with the name *dropout* by Geoffrey Hinton, et al. in 2012.<sup>[3]</sup> Google currently holds the patent for the dropout technique.<sup>[7][note 1]</sup>

## See also

---

- AlexNet
- Convolutional neural network § Dropout

## Notes

---

1. The patent is most likely not valid due to previous art. “Dropout” has been described as “dilution” in previous publications. It is described by Hertz, Krogh, and Palmer in *Introduction to the Theory of Neural Computation* (1991) ISBN 0-201-51560-1, pp. 45, *Weak Dilution*.

The text references Sompolinsky *The Theory of Neural Networks: The Hebb Rules and Beyond* in *Heidelberg Colloquium on Glassy Dynamics* (1987) and Canning and Gardner *Partially Connected Models of Neural Networks* in *Journal of Physics* (1988). It goes on to describe strong dilution. This predates Hinton's paper.

## References

---

1. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting" (<http://jmlr.org/papers/v15/srivastava14a.html>). *Jmlr.org*. Retrieved July 26, 2015.
2. Warde-Farley, David; Goodfellow, Ian J.; Courville, Aaron; Bengio, Yoshua (2013-12-20). "An empirical analysis of dropout in piecewise linear networks". arXiv:1312.6197 (<https://arxiv.org/abs/1312.6197>) [stat.ML (<https://arxiv.org/archive/stat>.ML)].
3. Hinton, Geoffrey E.; Srivastava, Nitish; Krizhevsky, Alex; Sutskever, Ilya; Salakhutdinov, Ruslan R. (2012). "Improving neural networks by preventing co-adaptation of feature detectors". arXiv:1207.0580 (<https://arxiv.org/abs/1207.0580>) [cs.NE ([https://arxiv.org/archive](https://arxiv.org/archive/cs)e/cs.NE)].
4. Hertz, John; Krogh, Anders; Palmer, Richard (1991). *Introduction to the Theory of Neural Computation*. Redwood City, California: Addison-Wesley Pub. Co. pp. 45–46. ISBN 0-201-51560-1.
5. Sompolinsky, H. (1987), "The theory of neural networks: The Hebb rule and beyond", *Heidelberg Colloquium on Glassy Dynamics*, Lecture Notes in Physics, vol. 275, Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 485–527, Bibcode:1987LNP...275..485S (<https://ui.adsabs.harvard.edu/abs/1987LNP...275..485S>), doi:10.1007/bfb0057531 (<https://doi.org/10.1007%2Fbfb0057531>), ISBN 978-3-540-17777-7
6. Canning, A; Gardner, E (1988-08-07). "Partially connected models of neural networks". *Journal of Physics A: Mathematical and General*. **21** (15): 3275–3284. Bibcode:1988JPhA...21.3275C (<https://ui.adsabs.harvard.edu/abs/1988JPhA...21.3275C>). doi:10.1088/0305-4470/21/15/016 (<https://doi.org/10.1088%2F0305-4470%2F21%2F15%2F016>). ISSN 0305-4470 (<https://search.worldcat.org/issn/0305-4470>).
7. US 9406017B2 (<https://patents.google.com/patent/US9406017B2/en>), Hinton, Geoffrey E., "System and method for addressing overfitting in a neural network", published 2016-08-02, issued 2016-08-02

---

Retrieved from "[https://en.wikipedia.org/w/index.php?title=Dropout\\_\(neural\\_networks\)&oldid=1360650719](https://en.wikipedia.org/w/index.php?title=Dropout_(neural_networks)&oldid=1360650719)"